

## Lab 3: Component Modelling & Architectural Pattern Selection

Written Justification – Self-Service Coffee Kiosk System

---

### Architecture Selection

We chose **Layered Architecture** for the **Self-Service Coffee Kiosk System**.

### Architectural Choice

Layered Architecture with three layers. The **Presentation Layer** holds the User Interface Component (touch screen); the **Business Layer** holds the Order Manager and Payment Service Components; the **Data & Device Access Layer** holds the Receipt Printer and Database Components. Each layer calls only the layer below it through the interfaces *IOrderService*, *IPaymentProcessing*, *IReceiptPrinting* and *IMenuData*, and the Payment Service reaches the external card gateway through *ICardAuthorization*.

### Two Reasons

1. **The scenario is a small, single-device system with one fixed workflow.** Every order follows the same path (choose one of 3 coffees, choose Small/Large, pay by credit card, print receipt), which maps directly onto a top-down flow from UI to business logic to data and hardware. Microservices would add deployment, networking and consistency overhead that a single kiosk with five components does not need, and a Client-Server design would make a walk-up kiosk in a busy café depend on a central server as a single point of failure.
2. **Clear separation of concerns makes each part replaceable.** The touch-screen handling, the ordering rules and the hardware/data access are isolated behind interfaces. A new receipt printer, a different payment provider, or a menu change (e.g. a third size) only affects one component, and the UI is untouched. Each component can also be tested on its own.

### Security Advantage

**Payment handling is confined to a single component.** Card details are handled only inside the Payment Service, which is the only component that talks to the external gateway (over HTTPS/TLS). The User Interface never sees or stores card data, and the Database holds only menu, pricing and order records, never card numbers. Because the layers forbid the UI from reaching the payment gateway, printer or database directly, every request must pass through the Order Manager. This gives one place to validate input and shrinks the attack surface and the amount of code that must meet payment-security requirements (PCI DSS).

### Performance Benefit

**Fast local calls keep the touch screen responsive.** All components run on the kiosk itself, so calls between the UI, Order Manager and local Database are in-process and add no network latency. Menu items and prices (only 3 coffees × 2 sizes) can be read from the local database or cached by the Order Manager, so screens update instantly. Only the card authorisation needs a network round trip. The small extra cost of passing a request through the layers is negligible at this scale.